

Latency-Aware Shard Placement for Distributed Vector Indexes

A. Ferreira¹, M. Okonkwo¹, R. Lindqvist², S. Bhatt¹

¹Department of Computer Science, Kestrel Institute of Technology ²Nordur Systems Research Lab

Abstract

Approximate nearest-neighbor (ANN) search over billion-scale vector collections is increasingly bottlenecked not by raw index throughput but by cross-shard network latency under skewed query load. We present LASP (Latency-Aware Shard Placement), a placement algorithm that co-locates frequently co-queried shards while respecting per-node memory budgets. On three production-scale workloads, LASP reduces p99 query latency by 31-44% relative to consistent-hashing placement, at the cost of a one-time offline rebalancing pass. We formalize the placement problem as a constrained graph partitioning instance and show it is NP-hard by reduction from balanced graph partitioning¹, then give a practical approximation with provable memory-budget guarantees.

Keywords: vector search, shard placement, graph partitioning, ANN indexes

1. Introduction

Vector search systems partition an index into shards distributed across many machines to fit within per-node memory limits. When a query fans out to k shards, the observed latency is dominated by the slowest responding shard plus the network round trip to whichever shards are not co-located with the querying node. Prior work has largely optimized shard *construction* (how vectors are clustered into shards) while treating shard *placement* (which physical node hosts which shard) as an afterthought handled by generic consistent hashing [3, 7].

We show that placement matters as much as construction once query patterns are skewed — a condition that holds in essentially every production workload we examined, where the top 5% of query clusters account for over 60% of traffic (Table 1, Section 4).

Our contributions are threefold. First, we formalize latency-aware shard placement as a constrained min-cut graph partitioning problem (Section 2) and prove NP-hardness. Second, we design LASP, a greedy local-search heuristic with a memory-budget-preserving swap operator (Section 3). Third, we evaluate LASP against three production workloads spanning e-commerce recommendation, semantic document search, and reverse image search (Section 4), showing consistent p99 latency reductions.

2. Problem Formulation

Let $G = (V, E, w)$ be a co-query graph where each vertex $v \in V$ represents a shard, and an edge $(i, j) \in E$ carries weight $w(i, j)$ equal to the number of queries in a representative trace that touch both shard i and shard j . Let each shard have a memory footprint $m(v)$, and let each of the N available nodes have a capacity budget B . A placement is a function $\pi: V \rightarrow \{1, \dots, N\}$ that assigns shards to nodes.

$$\text{cost}(\pi) = \sum_{(i,j) \in E, \pi(i) \neq \pi(j)} w(i, j) \quad (1)$$

subject to the per-node capacity constraint:

$$\sum_{v: \pi(v)=n} m(v) \leq B \quad \text{for all } n \in \{1, \dots, N\} \quad (2)$$

Equation 1 counts the total cross-node query weight that must cross the network; minimizing it is equivalent to maximizing the fraction of co-queried shard pairs placed on the same node. This is precisely the balanced graph partitioning objective with a hard capacity constraint rather than a balance constraint, which we show reduces from balanced graph partitioning (known NP-hard²) in Appendix A of the extended report³.

3. The LASP Algorithm

LASP proceeds in two phases. The *construction phase* builds an initial placement using a capacity-aware greedy graph growing procedure, seeded from the highest-weight edge in G . The *refinement phase* performs bounded local search: at each step, LASP considers swapping two shards between nodes if the swap strictly decreases $\text{cost}(\pi)$ without violating Equation 2 on either node, and applies the first improving swap found in randomized order (Algorithm 1).

Algorithm 1: LASP Refinement Phase

```
Require: co-query graph  $G$ , placement  $\pi_0$ , budget  $B$ , max_iters
1:  $\pi \leftarrow \pi_0$ 
2: for  $t = 1$  to max_iters do
3:    $(i, j) \leftarrow$  random edge in  $E$ 
4:   if swap( $i, j$ ) improves cost and satisfies Eq. 2 then
5:      $\pi \leftarrow$  apply swap( $i, j$ )
6:   end if
7: end for
8: return  $\pi$ 
```

We cap max_iters at $4N$ (four sweeps over the node count) in all experiments; Section 4.3 shows diminishing returns beyond this point across all three workloads.

3.1 Convergence Behavior

Because each accepted swap strictly decreases the non-negative integer cost function, the refinement phase is guaranteed to terminate. In practice it converges in far fewer than $4N$ iterations⁴; — see the side note for the empirical iteration counts we observed during development, prior to the formal evaluation in Section 4.

Side note

In early internal testing (not part of the formal evaluation), median convergence for a 512-shard graph occurred around 380 iterations,

well under the 2048-iteration cap (4×512). We do not include this internal run in the reported results because it used a synthetic co-query graph rather than production traces.

4. Evaluation

We evaluate LASP on three production-derived workloads, described in Table 1. Each workload was captured as a 7-day query trace from a live deployment and replayed offline against both a consistent-hashing baseline and LASP.

Workload	Shards	Nodes	Top-5% share
E-commerce recsys	2,048	96	61%
Semantic doc search	4,096	160	58%
Reverse image search	1,024	64	67%

Table 1. Workload characteristics used in evaluation.

4.1 Latency Results

Table 2 reports p50 and p99 query latency under both placement strategies. LASP reduces p99 latency by 31% on the e-commerce workload, 38% on semantic document search, and 44% on reverse image search — the largest gain occurring on the workload with the most concentrated query skew (67% top-5% cluster share, Table 1).

Workload	Baseline p99 (ms)	LASP p99 (ms)	Reduction
E-commerce recsys	184	127	31%
Semantic doc search	211	131	38%
Reverse image search	256	143	44%

Table 2. p99 latency before and after LASP rebalancing.

4.2 Rebalancing Cost

The one-time offline rebalancing pass took under 40 minutes for the largest workload (semantic document search, 4,096 shards) on a single rebalancing coordinator node, and does not require taking the serving cluster offline — shards are migrated one at a time with a brief dual-write window per shard.

4.3 Ablation: Iteration Cap

We swept `max_iters` from $0.5N$ to $8N$. Latency reduction plateaus beyond $4N$ across all three workloads (within 2 percentage points of the $8N$ result), which is why we fix `max_iters` = $4N$ as the default in Section 3.

4.4 Memory Overhead of the Co-query Graph

Storing G explicitly requires $O(|E|)$ memory, which for the semantic document search workload (4,096 shards) reached 1.8 GB when built from the full 7-day trace at full resolution. We found that sparsifying G by dropping edges with weight below the 90th percentile reduces this to 210 MB with less than 1%

degradation in final p99 latency reduction, and we use this sparsified variant in all reported results (Table 2).

FIGURE 1 — p99 Latency vs. Rebalancing Iteration

Figure 1. p99 latency after each refinement sweep, semantic document search workload. The curve flattens near iteration $4N$, matching the ablation in Section 4.3.

4.5 Sensitivity to Trace Staleness

Because LASP placement is derived from a historical query trace, a natural concern is whether placements degrade as live traffic drifts away from the trace used for rebalancing. We re-evaluated each placement against traffic captured 1, 7, and 30 days after the original trace window (Table 3).

Staleness	E-comm p99 (ms)	Doc search p99 (ms)	Image p99 (ms)
0 days (fresh)	127	131	143
1 day	129	133	145
7 days	138	142	158
30 days	167	179	201

Table 3. p99 latency as the query trace used for placement grows stale relative to live traffic.

By 30 days, latency has regressed to within 10-15% of the consistent-hashing baseline reported in Table 2, motivating the periodic rebalancing cadence discussed in Section 6.

4.6 Comparison with Alternative Heuristics

Beyond the consistent-hashing baseline, we compared LASP against two additional placement heuristics: (i) a spectral partitioning baseline using the normalized Laplacian of G , and (ii) a greedy bin-packing baseline that ignores co-query weight entirely and packs shards purely to balance memory usage. Table 5 reports p99 latency for all four strategies on the semantic document search workload.

Strategy	p99 latency (ms)	Rebalancing time
Consistent hashing (baseline)	211	n/a
Memory-balance bin-packing	198	< 5 min
Spectral partitioning	149	~ 55 min
LASP (this work)	131	< 40 min

Table 5. Comparison of placement strategies on the semantic document search workload.

Spectral partitioning achieves the second-best latency but at roughly 1.4x the rebalancing time of LASP, since computing eigenvectors of the (sparsified) graph Laplacian dominates its runtime at this scale; LASP's greedy swap search remains cheaper while reaching a lower final latency in our experiments.

4.7 Statistical Significance

Each latency figure in Tables 2, 3, and 5 is the median over 5 independent trace replays with different random seeds for tie-breaking in the greedy construction phase and the randomized swap order in refinement. The interquartile range across replays was under 4ms for every reported p99 figure, and a paired Wilcoxon signed-rank test comparing LASP to the consistent-hashing baseline was significant at $p < 0.01$ for all three workloads.

FIGURE 2 — Placement Comparison Across Strategies

Figure 2. p99 latency for all four placement strategies from Table 5, semantic document search workload, with interquartile ranges across the 5 trace replays described in Section 4.7.

5. Related Work

Consistent hashing [3] and rendezvous hashing [7] are the dominant placement strategies in production systems today, chosen primarily for their simplicity and stability under node churn rather than query-latency optimality. Graph partitioning for data placement has been studied extensively in the database literature [2, 5], but almost exclusively under a balance objective rather than a hard capacity constraint, and rarely with co-query-derived edge weights specific to vector search fan-out patterns.

Closer to our setting, HNSW-style graph indexes [6] optimize intra-shard traversal efficiency but are agnostic to which physical node hosts which portion of the graph; LASP is complementary and can sit on top of any shard construction method, including HNSW-based ones, since it only consumes the resulting co-query graph and per-shard memory footprints.

The classic Kernighan-Lin heuristic [4] for graph bisection inspired our swap-based local search, though Kernighan-Lin targets a balanced two-way cut rather than an N-way capacity-constrained assignment, requiring the capacity-aware modifications described in Section 3.

6. Discussion and Operational Guidance

Based on the staleness sensitivity results in Section 4.5, we recommend re-running the construction and refinement phases on a fresh trace weekly for workloads with volatile query patterns (e-commerce, driven by promotions and seasonality) and monthly for more stable workloads (reverse image search, in our deployment). Rebalancing can run incrementally: rather than recomputing π from scratch, the refinement phase alone can be re-run starting from the current placement as π_0 , which converges roughly 3x faster than cold-starting the construction phase.

A secondary operational benefit we did not anticipate at design time: because LASP explicitly models per-node capacity as a hard constraint (Equation 2), it naturally produces placements that leave a small amount of headroom on every node, which operators found useful for absorbing shard growth between rebalancing runs without manual intervention.

7. Limitations

LASP has three notable limitations. First, it requires a representative query trace; cold workloads with no query history default to consistent hashing until enough trace data accumulates, typically 24-48 hours in our deployments. Second, the refinement phase assumes shard memory footprints are roughly static between rebalancing runs; workloads with rapidly growing shards may need more frequent rebalancing than the cadence in Section 6. Third, our evaluation covers three workloads at one organization; we have not validated LASP against workloads with substantially different query fan-out characteristics, such as exhaustive multi-vector search.

8. Conclusion

We presented LASP, a latency-aware shard placement algorithm for distributed vector indexes, and showed 31-44% p99 latency reductions across three production-scale workloads. Future work includes extending LASP to handle online placement updates as query patterns drift, rather than requiring a full offline rebalancing pass, and validating the approach on workloads beyond the three studied here.

8.1 Engineering Checklist for Adopting LASP

Teams considering LASP for an existing deployment should work through the following checklist, grouped by phase:

1. Instrumentation
 - a. Confirm query logs record which shards each query touches
 - b. Confirm per-shard memory footprint is available or computable
2. Trace collection
 - a. Collect at least 7 days of representative traffic (Section 4.5)
 - b. Exclude known anomalous periods (e.g. incident-driven traffic spikes)
3. Rollout
 - a. Run construction + refinement offline and diff against current placement
 - b. Migrate shards incrementally with dual-write windows

(Section 4.2)

c. Verify per-shard replication factor is set independently of placement (Section 10)

4. Steady state

a. Schedule recurring refinement per the cadence guidance in Section 6

b. Alert if observed p99 drifts more than 20% above the post-rebalancing baseline, which in our experience indicates the trace has gone stale before the scheduled refinement (see Table 3)

8.2 Extended Evaluation on Synthetic Co-query Graphs

To test LASP outside the three production workloads in Section 4, we also generated synthetic co-query graphs using a power-law cluster model with tunable skew parameter α , sweeping α from 0.5 (near-uniform query distribution) to 2.5 (extremely concentrated, single-cluster-dominated traffic). LASP's advantage over consistent hashing grows monotonically with α : at $\alpha = 0.5$ the two strategies are statistically indistinguishable ($p > 0.3$), while at $\alpha = 2.5$ LASP achieves a 58% p99 reduction, higher than any production workload we measured, consistent with our hypothesis that LASP's benefit is driven specifically by exploitable query skew rather than by graph partitioning quality in general.

This synthetic sweep also lets us stress-test the capacity constraint in isolation: when we force B tight enough that fewer than 5% of nodes have any spare capacity, refinement-phase swap acceptance rate falls below 2%, and LASP's latency reduction degrades to within 5 percentage points of memory-balance bin-packing, since there is almost no freedom left to exploit co-query locality once capacity is this constrained.

9. Threats to Validity

Internal validity. Our latency measurements were collected on a shared internal cluster rather than dedicated hardware; to control for noisy-neighbor effects, we interleaved baseline and LASP runs rather than running them on separate days, and we report medians over 5 replays (Section 4.7) specifically to reduce sensitivity to transient cluster load.

External validity. All three workloads originate from a single organization's production traffic. Query skew of the magnitude we observe (58-67% of traffic in the top 5% of clusters, Table 1) is consistent with prior reports in the recommendation-systems literature, but we cannot rule out that organizations with flatter query distributions would see smaller gains from LASP, since the entire mechanism depends on exploitable skew.

Construct validity. We use p99 query latency as our primary metric because it is the metric our on-call teams alert on (Section 6 of the internal Orion administrator guide takes the same approach for a different system), but organizations optimizing for mean latency or total cluster cost rather than tail latency may reach different conclusions about whether the rebalancing overhead in Section 4.2 is worthwhile.

10. Broader Impact and Deployment Considerations

LASP's primary practical effect is to reduce the number of physical machines a fixed query-latency SLA requires, by making better use of existing cross-shard locality rather than by adding hardware. In our production deployment, the p99 reductions in Table 2 let the semantic document search workload meet its existing 150ms p99 SLA with roughly 15% fewer nodes than the consistent-hashing baseline required, which was the original motivation for this work rather than latency reduction as an end in itself.

We note one deployment caveat: because LASP concentrates co-queried shards onto fewer physical nodes than a uniformly-random placement would, it slightly increases the blast radius of a single node failure for the affected query clusters. Operators should ensure per-shard replication (orthogonal to placement, and outside the scope of this paper) is configured with a replication factor appropriate for their durability requirements before adopting LASP, since placement optimizes latency, not availability.

Appendix A — Sketch of the NP-Hardness Reduction

We reduce from Balanced Graph Partitioning (BGP), known NP-hard [1]. Given a BGP instance (graph H , k parts, balance factor), construct a LASP instance with:

- (a) co-query graph $G = H$, with edge weights copied directly from H ;
- (b) $N = k$ nodes, one per target partition;
- (c) uniform shard memory footprint $m(v) = 1$ for all v ;
- (d) capacity budget B set to the BGP balance factor times $|V(H)| / k$.

Any LASP placement satisfying Equation 2 under this construction corresponds exactly to a balanced k -way partition of H respecting the original balance factor, and $\text{cost}(\pi)$ equals the BGP cut weight. A polynomial-time LASP solver would therefore solve BGP in polynomial time, a contradiction. Full proof details are in the extended technical report³.

Appendix B — Hyperparameter Settings Used in Section 4

Parameter	Value
max_iters	4N
Edge sparsification percentile	90th
Swap selection	first-improving, randomized edge order
Rebalancing coordinator concurrency	8 parallel shard migrations
Dual-write window per shard	2-5 seconds, workload dependent

Table 4. Hyperparameters used for all results in Section 4 unless otherwise noted.

Appendix C — Notation Reference

Symbol	Meaning
$G = (V, E, w)$	Co-query graph; vertices are shards, edge weights are co-query counts
$m(v)$	Memory footprint of shard v
N	Number of physical nodes
B	Per-node capacity budget
π	A placement function $V \rightarrow \{1, \dots, N\}$
$\text{cost}(\pi)$	Total cross-node query weight under placement π (Equation 1)
max_iters	Refinement phase iteration cap, default $4N$

Table 6. Notation used throughout the paper.

References

- [1] Garey, M. and Johnson, D. Computers and Intractability. Freeman, 1979.
- [2] Karypis, G. and Kumar, V. A fast and high quality multilevel scheme for partitioning irregular graphs. SIAM J. Sci. Comput., 1998.
- [3] Karger, D. et al. Consistent hashing and random trees. STOC, 1997.
- [4] Kernighan, B. and Lin, S. An efficient heuristic procedure for partitioning graphs. Bell Sys. Tech. J., 1970.
- [5] Curino, C. et al. Schism: a workload-driven approach to database replication and partitioning. VLDB, 2010.
- [6] Malkov, Y. and Yashunin, D. Efficient and robust approximate nearest neighbor search using HNSW graphs. IEEE TPAMI, 2020.
- [7] Thaler, D. and Ravishankar, C. Using name-based mappings to increase hit rates. IEEE/ACM Trans. Networking, 1998.

¹See Appendix A of the extended technical report for the full reduction proof.

²Andreev and Racke (2006) show balanced graph partitioning is NP-hard even to approximate within a constant factor for general graphs.

³Extended report available at <https://research.kestrel.edu/lasp-extended>.
&sup4;N denotes the number of physical nodes, not the number of shards.